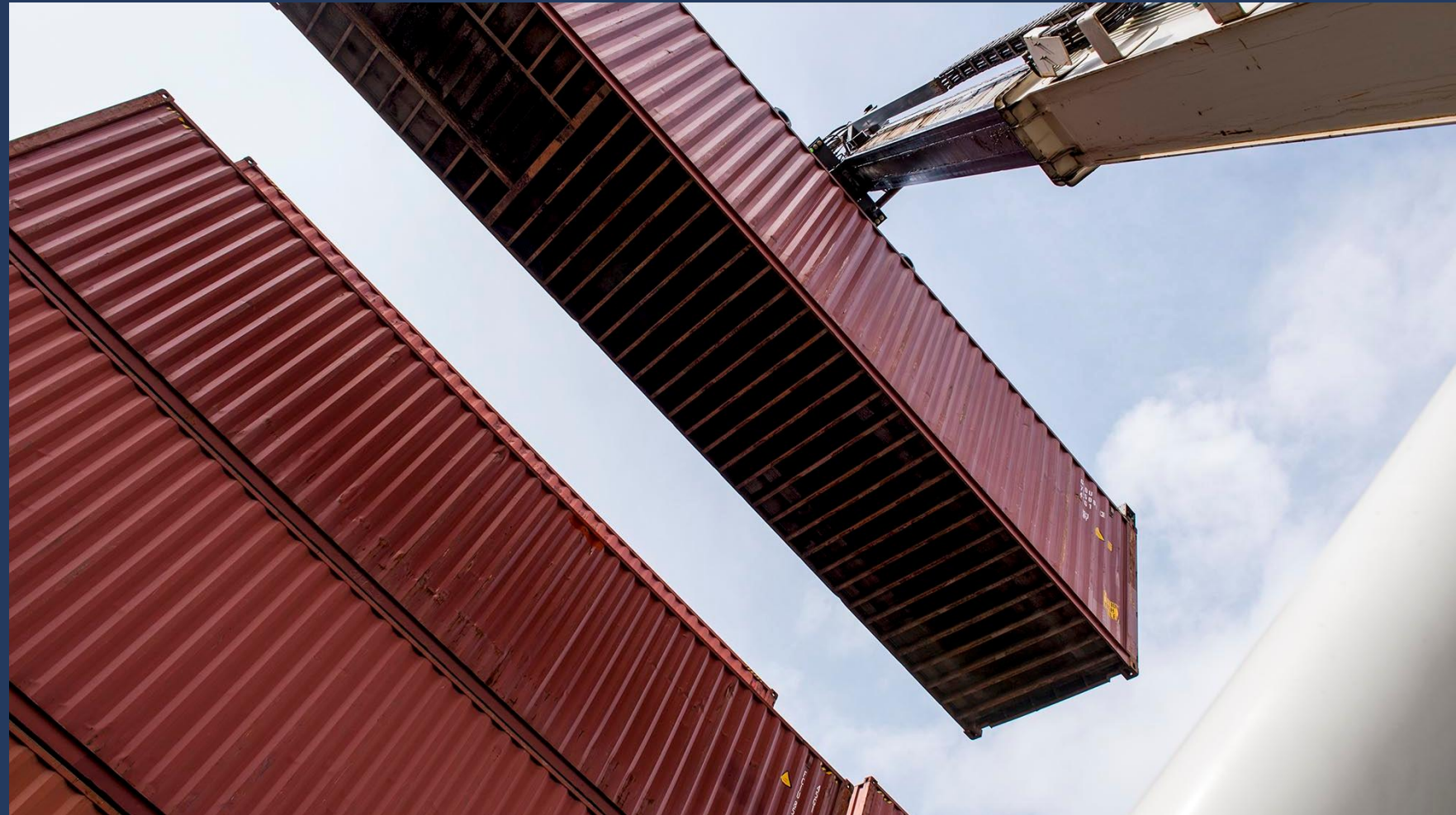


Kubernetes Technical Briefing

Module 4:
Application Deployment



Agenda

- DevOps and GitOps Concepts
- Continuous Integration Pipeline with Kubernetes
- Continuous Delivery Pipeline with Kubernetes

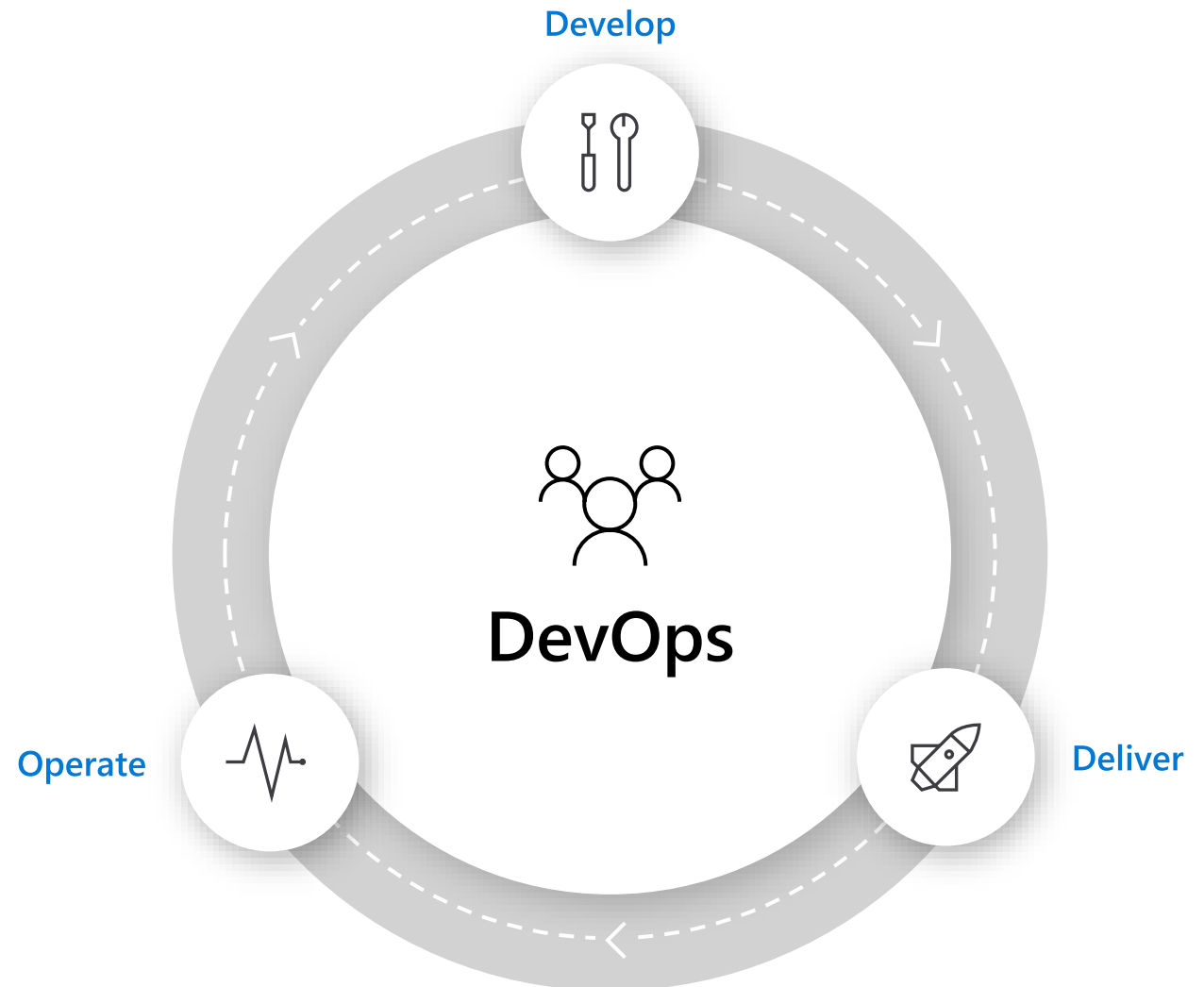
DevOps Concepts



What is DevOps?

- DevOps is the union of **people**, **process**, and **products** to enable continuous delivery of value to your end users.
- This training modules covers only basic functionality of tools like **Azure DevOps** and **GitHub**.
- Please refer to our other Microsoft DevOps workshops for more details.

People. Process. Products.

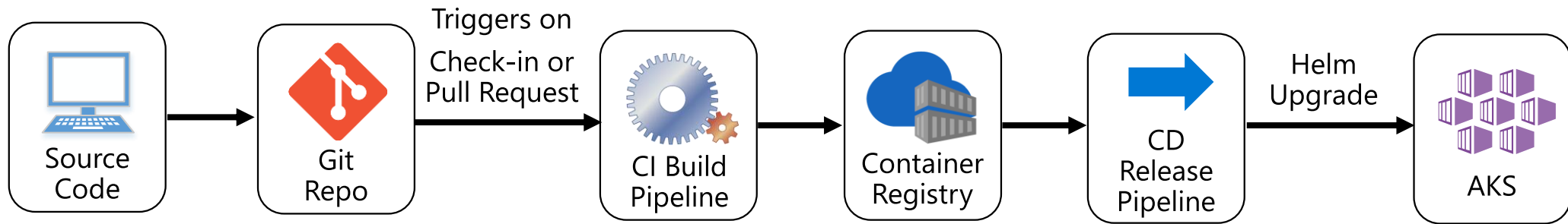


What is GitOps?

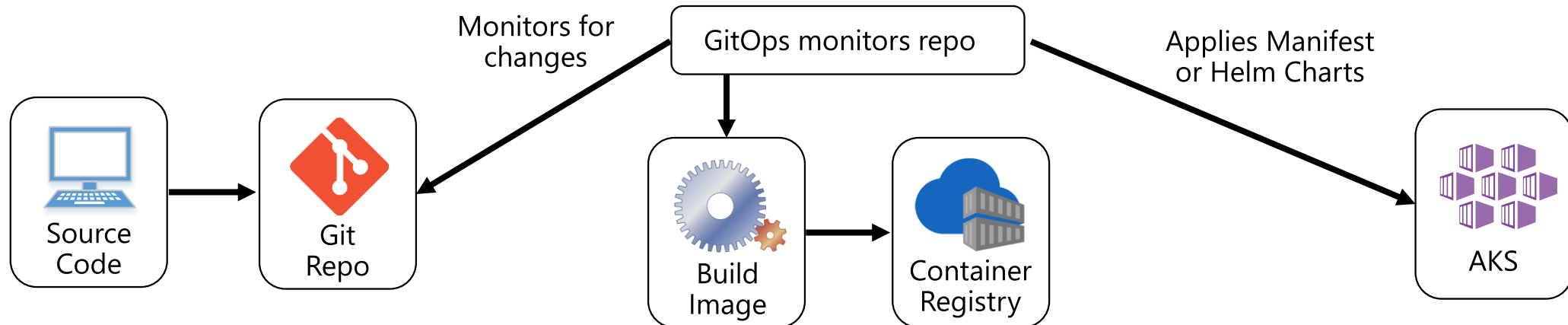
- GitOps is an operational framework that takes DevOps best practices used for application development such as version control, collaboration, compliance, and CI/CD, and applies them to infrastructure automation.
- GitOps uses a ***Git repository*** as the single source of truth for infrastructure definitions.
- GitOps automates infrastructure updates using a Git workflow with continuous integration (CI) and continuous delivery (CD).

DevOps vs GitOps

- **DevOps** uses pipelines that **Push** changes to a destination cluster.



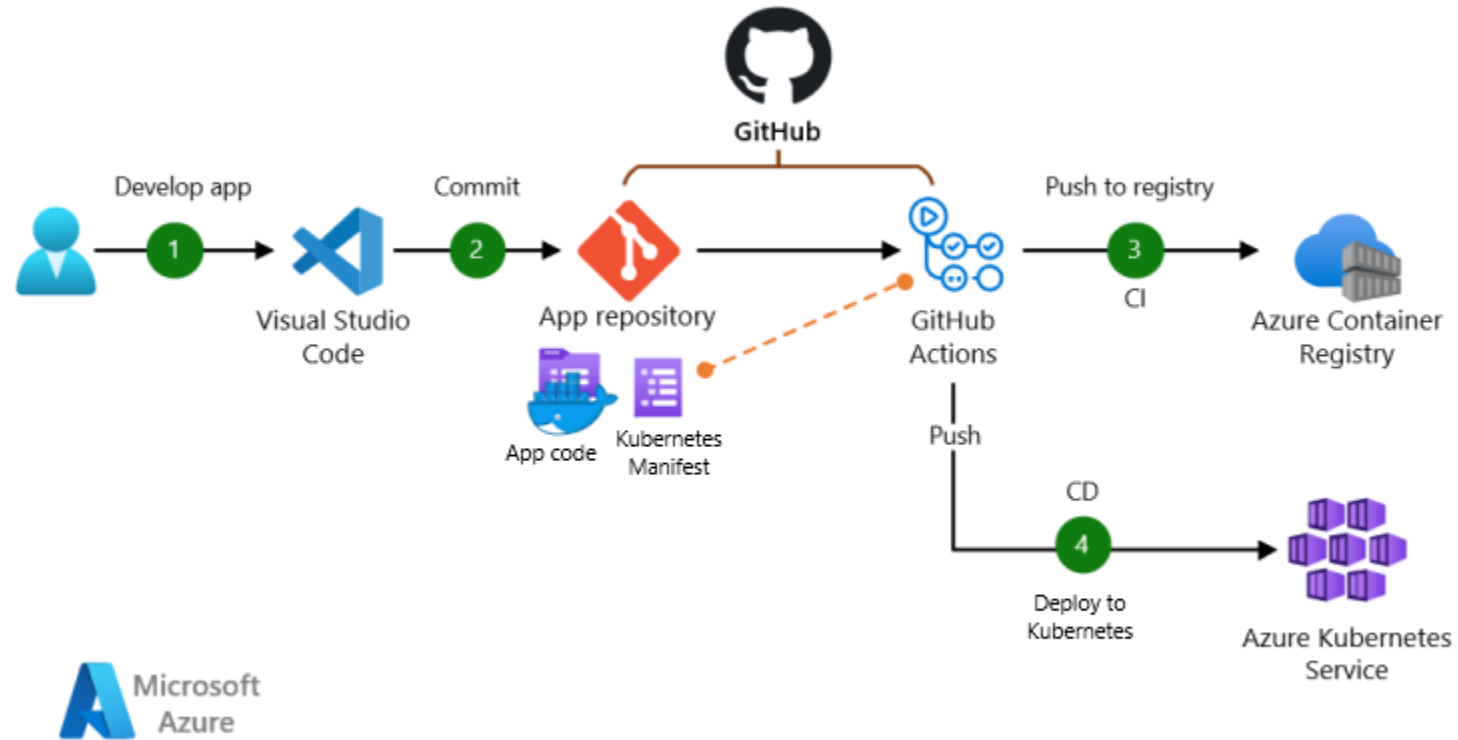
- **GitOps** monitors a Git repo and **Pulls** changes from the Git repo to synchronize a destination cluster.



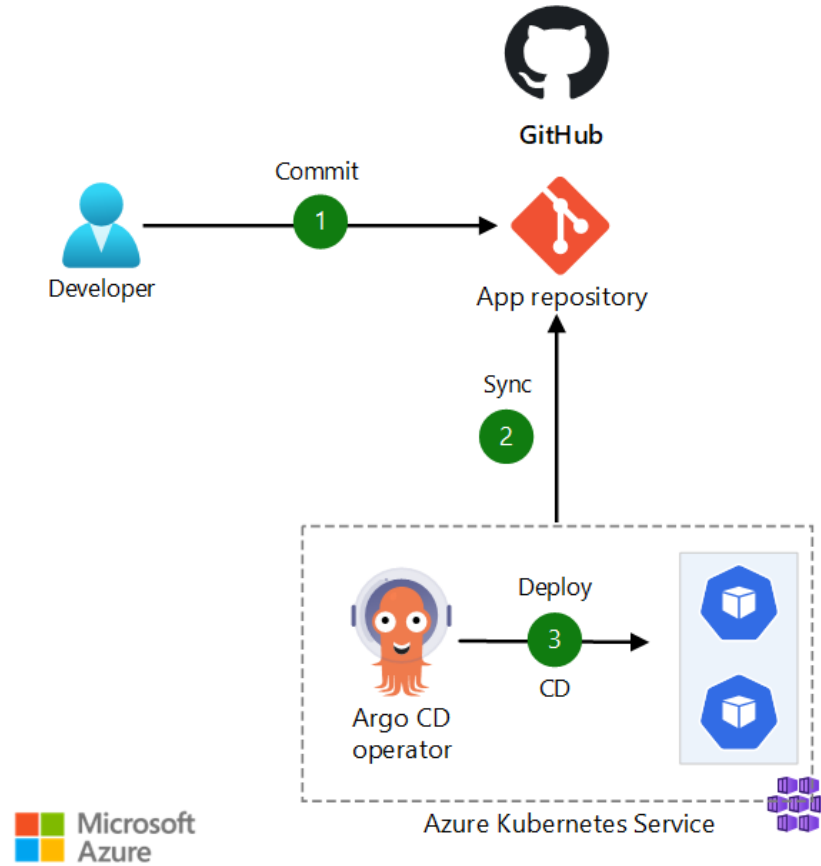
Repository Structure

- Application Repo
 - Application code resides here and Dockerfiles
 - Image building and pushing occurs here to ACR
- Deployment
 - Ideally a separate repo at a minimum can be a mono repo structure or polyrepo
 - Mono repo
 - Good for small projects with not a lot of environments
 - Can be harder to scale as projects grow
 - If using GitOps creates slower deployments by the GitOps Agents
 - Polyrepos
 - Multiple repos per service used in deployments

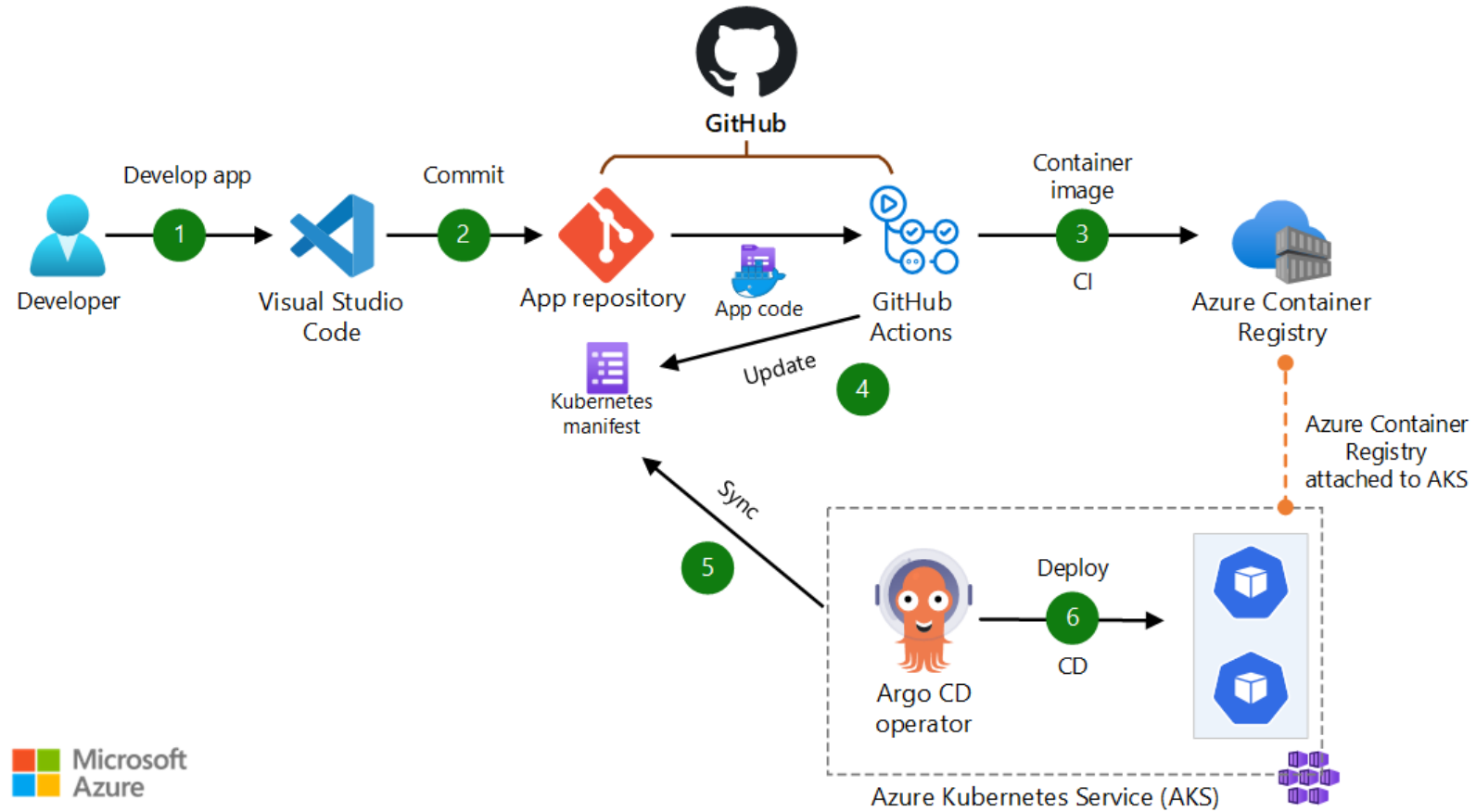
Git Hub CI/CD - Push



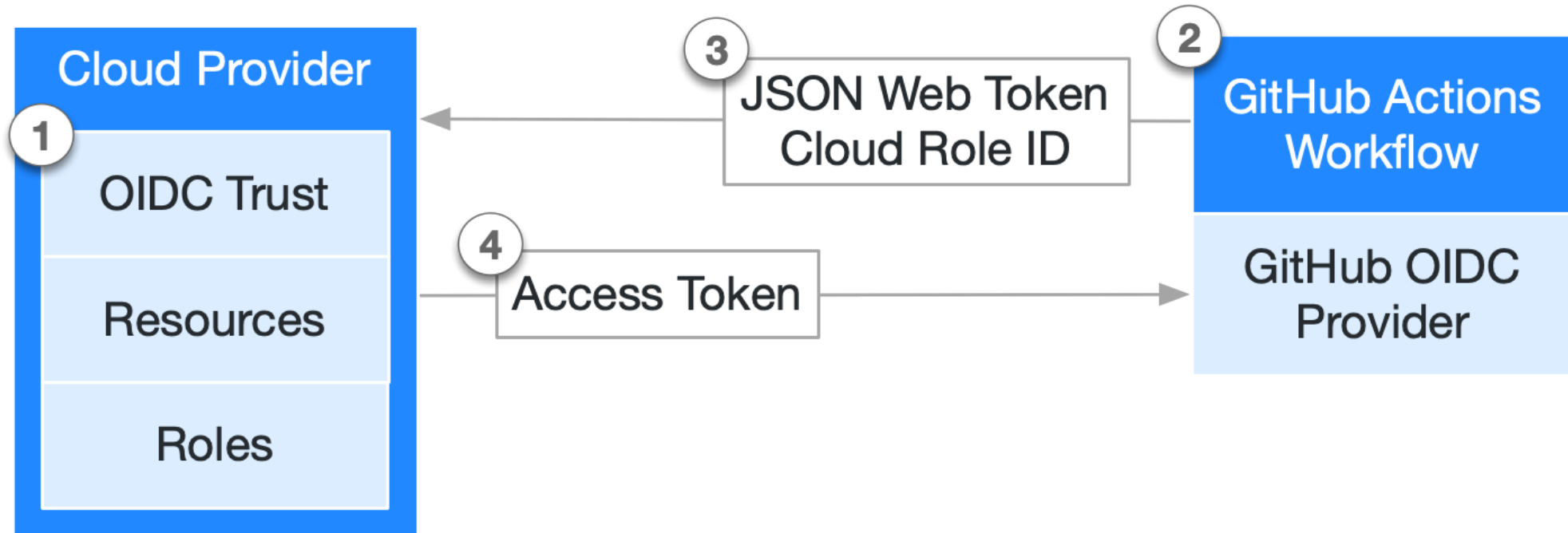
Git Hub CICD – Pull



Git Hub CI/CD – Pull



Git Hub Federate Identity



Continuous Integration with Kubernetes



Typical CI workflow

1. Developer uses an IDE, such as Visual Studio, to commit changes to GitHub or Azure DevOps.
2. A push to the backing git repo triggers a CI pipeline
3. CI Pipeline runs through standard linting, unit tests, SAST code scanning, secret scanning and DAST stages
4. Container dockerfile is linted for best practices
5. Pipeline then builds the container image using the dockerfile
6. Container is then built, stamped with a unique tag (typically the github sha) and pushed to a container registry to be consumed by the CD pipeline or GitOps agent

We are mainly concerned with the container stages here.

Image linting dockerfiles, building, and scanning

- Security is core piece of effective DevOps and GitOps practices.
- Incorporating a Dockerfile linting stage in the CI Pipeline is essential to finding issues within the file before the image is built
- Tools like [Dockle](#) can be used here to lint those files. This should run before an image build.
- After the image is built an image scanner should run in the pipeline to scan the completed image for issues present.
- Defender can be leveraged here or other 3rd parties or open-source alternatives like [Trivy](#)

CI Demo

Using GitHub CI/CD to build and push images to ACR

Continuous Delivery with Kubernetes and Helm



Complex Deployments with Helm

- Complex applications, that utilize multiple microservices, need a more detailed deployment process.
- Rather than updating individual deployments using “**kubectl apply**”, the CD process uses **Helm** to upgrade the application as a complete system.
- If the “latest” version of each microservice is deployed, then Helm can simply do an upgrade without any changes.
- However, it’s best practice not to use the “latest” tag of any container (public or custom) in a production application.
- To correctly upgrade a chart, Helm needs to know the specific version of each microservice to deploy.
- Staging and production pipelines usually need to be able to deploy different versions of each microservice.

Microservice Azure DevOps CI/CD Process

The microservice DevOps CI/CD process consists of the following parts:

- **Code Repository** – Contains the microservice source code.
- **Build Pipeline** – Triggers when source code is checked in or a pull request is made.
 - A Build Number is used to uniquely identify the image about to be built. Can be built-in or generated.
 - A Docker *build* task builds an image based on the Dockerfile included with the source code. It uses the Build Number (or variation of it) as the image's tag.
 - A Docker *push* task pushes the image to a container registry.
- **Release Pipeline** – Triggers when ANY associated Build completes
 - A RegEx task updates all the tags in a copy of the **values.yaml** file.
 - A Helm task uses the updated values file to upgrade the chart:

```
helm upgrade myrelease mychart --install -f updatedvalues.yaml
```

CD Demo

Using GitHub CI/CD to deploy helm chart to AKS

GitOps Considerations

- Helm Deployments
 - Need to overwrite values.yaml for the image tags that are in ACR
 - Repos with values.yaml per environment might be used here
- [Published AKS GitOps Scenarios](#)

Lab – Module 4

Application Deployment to Kubernetes





Thank you